编号：HY-MS-OPD-T13

版本： V1.0

发布日期：2011-1-5

数据库设计说明书

项目名称： “三重一大”决策与运行管理系统

委托单位： 中国石油大学（北京）克拉玛依校区

承担单位： 短学期 23 小组

编写：祝梓熙 2025 年 6 月 21 日

校对：方叶武 2025 年 6 月 21 日

审核：梁天乐 2025 年 6 月 21 日

PPQA：方叶武 2025 年 6 月 21 日

目 录

1.引言 **4**

1.1 编写说明 4
1.2 背景 4
1.3 修订审批记录 4
1.4 术语和缩写词 5
1.5 参考资料 6

2 外部设计 **6**

2.1 标识符和状态 6
2.2 使用该数据库的程序 6
2.3 约定 7
2.3.1 数据库中表（Table）的命名约定 *7*
2.3.2 数据库表中字段（表的域 Column）命名约定 *8*
2.3.3 数据库中存储过程（Store Procedure）命名约定 *10*
2.3.4 数据库中触发器（Trigger）命名约定 *11*
2.3.5 数据库中游标（Cursor）命名约定 *12*
2.3.6 数据库索引（Index）命名约定 *12*
2.3.7 数据库中创建的视图（View）命名约定 *14*
2.3.8 数据库中函数（FUNCTION）命名约定 *14*
2.3.9 数据库约束（Constraint）命名约定 *15*
2.3.10 输入参数及输出参数命名约定 *16*
2.5 支持软件 17
2.6 专门说明 17

3 结构设计 **18**

3.1 概念结构设计 18
3.2 逻辑结构设计 31
3.3 物理结构设计 38
3.3.1 设计数据的存取路径 *38*
3.3.2 设计数据的存放位置 *39*

4、其它设计 **39**

4.1 完整性设计 39
4.2 安全保密设计 43
4.2.1. 数据库安全保密的设计考虑 *43*
4.2.2. 保证数据安全保密的措施和机制 *44*
4.3 其它设计 ..... ***错误！未定义书签。***

附件 1 数据库逻辑结构字典格式 **47**

1.用户服务数据库 (USER_SERVICE) 47
2.新闻服务数据库 (NEWS-SERVICE) 48
3.会议服务数据库 (MEETING_SERVICE) 51
4. 提案服务数据库 (PROPOSAL_SERVICE) 56

1. 引言

1.1 编写说明

《数据库设计说明书》的编制，给系统设计、系统开发提供参考，主要读者是系统设计人员、系统开发人员、用户。

1.2 背景

说明待开发数据库的名称、版本号说明、使用范围并列出本项目的任务提出者和开发者。

项目名称：“三重一大”决策与运行管理系统

委托单位：中国石油大学（北京）克拉玛依校区

开发单位：中国石油大学（北京）克拉玛依校区

用　　户：中国石油大学（北京）克拉玛依校区机关单位

数据库名称：“三重一大”决策与运行管理系统数据库

1.3 修订审批记录

说明编写这份《数据库设计说明书》的修订过程、审批过程。参见文档修订记录表及文档审批记录表。

表 1 文档修订记录表

修订记录					
章节	修订日期	版本	修订描述	修订者	审核者
1	2025/6/20	1.0	初稿第一版	祝梓熙	方叶武
2	2025/6/22	1.1	完善第一版	祝梓熙	方叶武
3	2025/6/25	1.2	格式更正	祝梓熙	方叶武
4	2026/7/2	1.3	终版	祝梓熙	方叶武

表 2 文档审批记录表

审批记录					
提交者	审批方式	审批日期	版本	审批意见	审批者
祝梓熙	审批	2025/7/1	1.2	数据字典不符合表命名约定	方叶武
祝梓熙	审批	2025/7/2	1.3	通过	方叶武

1.4 术语和缩写词

概念结构设计：是对现实世界的一种抽象，即对实际的人、物、事和概念进行人为处理，抽取人们关心的共同特性，忽略非本质的细节，并把这引起特性用各种概念精确地加以描述。概念结构独立于数据库逻辑结构，也独立于支持数据库的DBMS。

逻辑数据库设计：是根据用户要求和特定数据库管理系统的具体特点，以数据库设计理论为依据，设计数据库全局逻辑结构和每个用户的局部逻辑结构。

物理数据库设计：是在逻辑结构确定之后，设计数据库的存储结构及其它细节。

数据字典：是存储“关于数据项的数据”，它记录有关数据的来源、说明、与其他数据的关系、用途和格式等信息。

下面是ORACLE数据库专用术语和名称：

1. 表 ---------- Table
2. 字段 ---------- Column
3. 存储过程 ---------- Stored Procedure
4. 触发器 ---------- Trigger
5. 游标 ---------- Cursor
6. 索引 ---------- Index
7. 视图 ---------- View
8. 函数 ---------- Function
9. 约束 ---------- Constraint

SQL (Structured Query Language / 结构化查询语言)：用于与关系型数据库通信和操作的标准化语言。

DDL (Data Definition Language / 数据定义语言)：SQL 命令的一个子集，用于定义数据库结构，例如 CREATE TABLE（创建表）、ALTER TABLE（修改表）、DROP TABLE（删除表）。

DML (Data Manipulation Language / 数据操作语言)：SQL 命令的一个子集，用于管理模式对象中的数据，例如 INSERT（插入）、UPDATE（更新）、DELETE（删除）。

DCL (Data Control Language / 数据控制语言)：SQL 命令的一个子集，用于控制对数据库中数据的访问，例如 GRANT（授权）、REVOKE（撤销权限）。

TCL (Transaction Control Language / 事务控制语言)：SQL 命令的一个子集，用于管理数据库中的事务，例如 COMMIT（提交）、ROLLBACK（回滚）、SAVEPOINT（保存点）。

1.5 参考资料

《“三重一大”决策与运行管理系统软件需求说明书》

《“三重一大”决策与运行管理系统系统设计说明书》

2 外部设计

2.1 标识符和状态

本文中数据库采用 MySQL 数据库管理系统，在系统中使用 root 用户进行管理和维护，数据库名称是 “三重一大”决策与运行管理系统数据库。

程序名称： Navicat 版本号： 16.0

程序名称： Microsoft Office Excel 版本号： 2019

2.2 使用该数据库的程序

“三重一大”决策与运行管理系统-会议管理系统

版本号： 1.0

“三重一大”决策与运行管理系统-审批管理系统

版本号： 1.0

“三重一大”决策与运行管理系统-综合管理系统

版本号： 1.0

“三重一大”决策与运行管理系统-新闻展示系统

版本号： 1.0

2.3 约定

叙述使用该数据库所必须了解的建立标号、标识的有关约定。例如，用于标识库内各个文卷、记录、数据项的命名约定等。

2.3.1 数据库中表（Table）的命名约定

业务模块前缀 + 实体名：

- **约定形式：**模块缩写/业务领域_实体名
- **说明：**表名以其所属的业务模块或领域进行缩写开头，后跟该表所代表的实体名称。这种方式清晰地指明了表的归属和内容，方便管理和理解。
- **示例：**
  - **用户服务相关数据：**
    - user_info：用户基本信息。
  - **会议服务相关数据：**
    - meeting_conferee：会议参会人员信息。
    - meeting_file：会议相关文件信息。
    - meeting_info：会议基本信息。
    - meeting_summary：会议总结信息。
  - **提案服务相关数据：**
    - proposal_approval：提案审批信息。
    - proposal_file：提案相关文件信息。
    - proposal_info：提案基本信息。
  - **新闻服务相关数据：**
    - carousel_image：前端轮播图信息。
    - news：新闻展示信息。

副本/备份表命名：

- **约定形式：**原表名_copyX (其中 X 为数字)

- **说明**：用于表示某个表的副本或临时备份。
- **示例**：
  - user_info_copy1
  - user_info_copy2

2.3.2 数据库表中字段（表的域 Column）命名约定

基本约定：字段名通常采用**有意义的英文单词或其常用缩写**，并使用**小写字母**，多个单词之间用**下划线 _ 分隔**。

主要类别与示例：

- **标识符/ID**：通常使用 id 或 xxx_id 作为主键或关联键，其中 xxx 代表所属实体。
  - conferee_id (参会人员 ID)
  - meeting_id (会议 ID)
  - file_id (文件 ID)
  - meetint_id (会议 ID)
  - summary_id (总结 ID)
  - id (用户 ID)
  - proposal_id (提案 ID)
  - approval_id (审批 ID)
  - carousel_id (轮播图 ID)
  - news_id (新闻 ID)
- **名称/标题**：通常使用 name 或 title，或 xxx_name/xxx_title。
  - meeting_name (会议名称)
  - file_name (文件名称)
  - name (姓名)
  - proposal_name (提案名称/标题)
  - title (轮播图标题)
- **描述/内容/摘要**：通常使用 content 或 abstract，或 xxx_content/xxx_abstract。
  - meeting_content (会议内容)
  - summary_content (总结内容)
  - proposal_abstract (提案摘要)

- description (图片描述)

- **时间/日期**：通常使用 time 或 date，或 xxx_time/created_at/updated_at 等.
  - meeting_start_time (会议开始时间)
  - meeting_end_time (会议结束时间)
  - approval_time (审批时间)
  - proposal_start_time (提案开始/提交时间)
  - proposal_end_time (提案结束时间)
  - created_time (创建时间)
  - updated_time (更新时间)
  - created_at (记录创建时间)
  - updated_at (最后更新时间)
- **状态/类型/级别/优先级**：通常使用 status, type, level, priority 等。
  - file_type (文件类型)
  - meeting_status (会议状态)
  - meeting_type (会议类型)
  - is_admin (是否管理员)
  - department (部门)
  - approval_level (审批级别)
  - approval_status (审批状态)
  - proposal_type (提案类型)
  - proposal_level (提案审批级别)
  - proposal_priority (提案优先级)
  - proposal_status (提案状态)
  - status (轮播图状态)
  - display_status (新闻展示状态)
- **关联信息**：通常使用 user_id, department 等表示关联的用户或部门。
  - user_id (参会人员 ID)
  - meeting_owner (会议发起人/提案发起人)
  - metting_department (会议所属部门)
  - approval_department (审批部门)

- approval_user (审批人 ID)
- updated_by (最后操作人)

冲突处理：从提供的 SQL 文件来看，即使某些字段可能存在拼音缩写相同的情况，项目实际中通过加入**英文前缀**或**完整英文单词**来明确区分字段含义，而非严格遵循拼音缩写冲突处理规则。例如，meeting_id 和 proposal_id 都包含 id，但通过前缀清晰地区分了其所属实体。

2.3.3 数据库中存储过程（Store Procedure）命名约定

数据统计的存储过程名：以 KSP_ + 统计表表名组成。

- **示例**：
  - KSP_proposal_info_summary (统计提案信息的存储过程)
  - KSP_meeting_attendance_stats (统计会议出席情况的存储过程)

数据检查的存储过程名：以 SJJC_ + 表名 + 所要进行检查的相关字段名组成。

- **示例**：
  - SJJC_user_info_phone (检查用户信息表中电话字段的存储过程)
  - SJJC_proposal_approval_status (检查提案审批状态的存储过程)

查询依据的存储过程名：以 CYYJ_ + 表名 + 所要进行检查的相关字段名组成。

- **示例**：
  - CYYJ_proposal_info_type (查询提案信息依据类型的存储过程)
  - CYYJ_meeting_info_owner (查询会议信息依据发起人的存储过程)

报表处理的存储过程名：以 BB_ + 报表名称 + 序号组成。

- **示例**：
  - BB_ProposalApprovalReport_01 (提案审批报表处理存储过程 1)
  - BB_MeetingSummaryReport_01 (会议总结报表处理存储过程 1)

生成树的过程名：以 TREE_ + 表名组成。

- **示例**：
  - TREE_department (生成部门树的存储过程，如果存在部门层级表)

信息查询的存储过程名：XXCX_ + 业务名称拼音缩写 + 序号组成。

- **基于当前系统模块的示例**：
  1. **会议管理数据**：XXCX_HYGL_01 (查询会议管理信息存储过程 1)

2. **审批管理数据 (提案审批)**：XXCX_SPGL_01 (查询审批管理信息存储过程 1)

3. **综合管理数据 (用户/系统通用)**：XXCX_ZHGL_01 (查询综合管理信息存储过程 1)

4. **新闻展示数据**：XXCX_XWZS_01 (查询新闻展示信息存储过程 1)

2.3.4 数据库中触发器（Trigger）命名约定

插入触发器：名称以 INS_ + 当前表名组成。

- **示例**：
  - INS_user_info (在 user_info 表上插入数据时触发)
  - INS_proposal_info (在 proposal_info 表上插入数据时触发)
  - INS_meeting_conferee (在 meeting_conferee 表上插入数据时触发)
  - INS_news (在 news 表上插入数据时触发)

更新触发器：名称以 UPD_ + 当前表名组成。

- **示例**：
  - UPD_user_info (在 user_info 表上更新数据时触发)
  - UPD_proposal_approval (在 proposal_approval 表上更新数据时触发)
  - UPD_meeting_info (在 meeting_info 表上更新数据时触发)
  - UPD_carousel_image (在 carousel_image 表上更新数据时触发)

删除触发器：名称以 DEL_ + 当前表名组成。

- **示例**：
  - DEL_user_info (在 user_info 表上删除数据时触发)
  - DEL_proposal_file (在 proposal_file 表上删除数据时触发)
  - DEL_meeting_file (在 meeting_file 表上删除数据时触发)
  - DEL_news (在 news 表上删除数据时触发)

其它功能的触发器：应用最能表现此触发器功能的字母名称表示。

- **示例**：
  - LOG_user_activity (记录用户活动日志的触发器)
  - CHK_meeting_time_overlap (检查会议时间是否重叠的触发器)

2.3.5 数据库中游标（Cursor）命名约定

在数据库中创建的游标（Cursor），均以 CUR_为前缀，之后加最能表现此游标（Cursor）功能的术语或名称的汉语拼音缩写命名。

- **约定形式**：CUR_ + 功能或用途的汉语拼音缩写
- **示例 (假定)**：
  - CUR_HYLB (用于遍历会议列表的游标，"会议列表"的拼音缩写)
  - CUR_SPMX (用于遍历审批明细的游标，"审批明细"的拼音缩写)
  - CUR_YHLB (用于遍历用户列表的游标，"用户列表"的拼音缩写)
  - CUR_XWCK (用于遍历新闻查看的游标，"新闻查看"的拼音缩写)
  - CUR_BBJS (用于报表计算的游标，"报表计算"的拼音缩写)

2.3.6 数据库索引（Index）命名约定

主键索引 (Primary Key Index)：

- **约定形式**：通常没有显式命名，而是由数据库系统自动生成，名称通常为 PRIMARY。
- **说明**：在 CREATE TABLE 语句中通过 PRIMARY KEY (column_name) 定义。
- **示例**：
  - PRIMARY KEY (\approval_id`) USING BTREEinproposal_approval` table
  - PRIMARY KEY (\file_id`) USING BTREEinproposal_file` table
  - PRIMARY KEY (\proposal_id`) USING BTREEinproposal_info` table
  - PRIMARY KEY (\id` DESC) USING BTREEinuser_info` table
  - PRIMARY KEY (\conferee_id`) USING BTREEinmeeting_conferee` table
  - PRIMARY KEY (\file_id`) USING BTREEinmeeting_file` table
  - PRIMARY KEY (\meetint_id`) USING BTREEinmeeting_info` table
  - PRIMARY KEY (\summary_id`) USING BTREEinmeeting_summary` table
  - PRIMARY KEY (\carousel_id`) USING BTREEincarousel_image` table
  - PRIMARY KEY (\news_id`) USING BTREEinnews` table

普通索引 (Non-Unique Index)：

- **约定形式**：以 idx_ 为前缀，后接能表示索引主要作用的字段名或组合字段名，通常是下划线分隔的小写单词。

- **说明**：用于提高查询性能，不强制字段值的唯一性。
- **示例**：
    - idx_level (在 proposal_approval 表上，基于 approval_id 和 approval_level)
    - idx_department (在 proposal_approval 表上，基于 approval_department 和 approval_status)
    - idx_user (在 proposal_approval 表上，基于 approval_user)
    - idx_proposal (在 proposal_file 表上，基于 proposal_id)
    - idx_owner (在 proposal_info 表上，基于 proposal_owner)
    - idx_status (在 proposal_info 表上，基于 proposal_type, proposal_start_time, proposal_level, proposal_priority, proposal_status 组合)
    - idx_department_admin (在 user_info 表上，基于 is_admin 和 department 组合)
    - idx_conferee (在 meeting_conferee 表上，基于 meeting_id 和 user_id 组合)
    - idx_file (在 meeting_file 表上，基于 meeting_id 和 file_type 组合)
    - idx_proposal (在 meeting_info 表上，基于 proposal_id 和 meeting_type 组合)
    - idx_owner (在 meeting_info 表上，基于 meeting_owner 和 metting_department 组合)
    - idx_summary (在 meeting_summary 表上，基于 meeting_id 和 approval_id 组合)
    - idx_news_status (在 news 表上，基于 display_status)
    - idx_news_sort (在 news 表上，基于 sort_order)

唯一索引 (Unique Index)：

- **约定形式**：以 uk_ 为前缀，后接能表示索引唯一性约束的字段名或组合字段名，通常是下划线分隔的小写单词。
- **说明**：强制字段值的唯一性。
- **示例**：
    - uk_news_proposal (在 news 表上，基于 proposal_id 确保提案 ID 的唯一性)

2.3.7 数据库中创建的视图（View）命名约定

在数据库中创建的视图（View），均以基表 + _V 组成。

- **约定形式**：基表名_V
- **示例**：
    - user_info_V (基于 user_info 表的视图，用于筛选特定用户或隐藏敏感信息)
    - proposal_info_V (基于 proposal_info 表的视图，用于简化查询提案摘要信息)
    - meeting_info_V (基于 meeting_info 表的视图，用于提供会议概览信息)
    - news_V (基于 news 表的视图，用于前端展示特定新闻内容)
    - proposal_approval_V (基于 proposal_approval 表的视图，用于查看特定审批流程的汇总信息)

2.3.8 数据库中函数（FUNCTION）命名约定

在数据库中创建的函数（FUNCTION）命名规则，通常旨在清晰地表达函数的功能和返回结果。

1. **功能前缀约定**：
    - **约定形式**：通常以 fn_ 或 func_ 为前缀，表明这是一个自定义函数。
    - **说明**：这有助于将自定义函数与内置函数区分开来。
    - **示例**：
        - fn_calculate_meeting_duration (计算会议时长的函数)
        - func_get_user_department_name (根据用户 ID 获取部门名称的函数)
2. **功能描述命名**：
    - **约定形式**：前缀后，使用**动词+名词**的形式，或直接使用**描述函数功能的英文单词或其拼音缩写**，采用小写字母和下划线分隔。
    - **说明**：名称应简洁明了，准确反映函数执行的操作或返回的值。
    - **示例**：
        - fn_get_proposal_status_text (获取提案状态文本描述的函数)
        - fn_is_admin_user (判断用户是否为管理员的函数，通常以 is_ 或 has_ 开头用于布尔返回值的函数)

- fn_format_time_string (格式化时间字符串的函数)
- fn_count_active_meetings (统计活跃会议数量的函数)

2.3.9 数据库约束（Constraint）命名约定

在 MySQL 数据库中，约束的命名约定主要体现在其类型和作用上。

1. **主键约束 (Primary Key Constraint)**:
   - **约定形式**：通常**不显式命名**。MySQL 会自动为主键约束及其关联的唯一索引命名为 PRIMARY。这是 MySQL 的默认行为，并且不建议手动更改或使用 PRIMARY 作为其他索引或约束的名称.
   - **说明**：通过在列定义后或表定义末尾使用 PRIMARY KEY 关键字来指定。主键列隐含地为 NOT NULL.
   - **示例**：

```
CREATE TABLE `user_info` (
  `id` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL COMMENT '主键 登录',
  -- ... other columns
  PRIMARY KEY (`id` DESC) USING BTREE -- Constraint name is implicitly 'PRIMARY'
) ENGINE = InnoDB;
```

2. **唯一约束 (Unique Constraint)**:
   - **约定形式**：在您的 SQL 文件中，唯一性通常通过**唯一索引**来实现，其命名约定与普通索引类似，前缀为 uk_，后跟表名和字段名（或组合字段名）。
   - **说明**：确保指定列或列组合中的所有值都是唯一的。与主键不同，唯一约束允许 NULL 值（但 NULL 值本身也必须是唯一的，如果有多个 NULL 值则取决于数据库的具体实现）。
   - **示例**：

```
CREATE TABLE `news` (
  `news_id` int NOT NULL AUTO_INCREMENT COMMENT '表主键，自增',
  `proposal_id` int NOT NULL COMMENT '原提案 ID，外键',
  -- ... other columns
```

PRIMARY KEY (`news_id`) USING BTREE,

UNIQUE INDEX `uk_news_proposal`(`proposal_id` ASC) USING BTREE -- Explicitly named unique index

) ENGINE = InnoDB;

3. **外键约束 (Foreign Key Constraint)**:
   - **约定形式**：您的 SQL 文件**没有显式定义外键约束的名称**，而是直接在 CREATE TABLE 语句中通过 FOREIGN KEY (col_name) REFERENCES tbl_name (col_name) 来定义。在这种情况下，MySQL 会自动生成一个默认名称（通常是 表名_ibfk_1, 表名_ibfk_2 等）。
   - **行业常见约定**（未在您文件中体现但推荐的做法）：通常以 fk_ 为前缀，后跟源表名和目标表名，例如 fk_source_table_target_table. 显式命名外键有助于在违反约束时提供更清晰的错误信息.
   - **说明**：用于强制两个表之间列的链接，确保数据引用完整性。
4. **非空约束 (NOT NULL Constraint)**:
   - **约定形式**：通常**不显式命名**。直接在列的数据类型后面加上 NOT NULL 关键字。MySQL 不会为 NOT NULL 约束自动生成可查看的名称，它只是列定义的一部分.
   - **说明**：确保列不能包含 NULL 值.
   - **示例**：

CREATE TABLE `user_info` (

`id` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL COMMENT '主键 登录',

`title` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL COMMENT '轮播图标题',

-- ... other columns

) ENGINE = InnoDB;

2.3.10 输入参数及输出参数命名约定

约定形式：P_ + 参数的每一个汉字拼音的第一个字母组合。

说明：这种命名方式旨在通过拼音缩写来简洁地表示参数的含义，并且通过 P_ 前缀清晰地标识其为参数。

示例：

- **输入参数**：
  - P_HYID (表示“会议 ID”的输入参数)
  - P_SPJG (表示“审批结果”的输入参数)
  - P_YHID (表示“用户 ID”的输入参数)
  - P_XWBT (表示“新闻标题”的输入参数)
  - P_QSSJ (表示“起始时间”的输入参数)
- **输出参数**：
  - P_ZCS (表示“总次数”的输出参数)
  - P_JGDM (表示“结果代码”的输出参数)
  - P_JGFW (表示“结果范围”的输出参数)

2.5 支持软件

“三重一大”决策与运行管理系统应用数据库采用 MySQL 8.0 进行数据维护、数据库备份、恢复、权限管理等。

数据库备份命令：mysqldump -u [username] -p[password] [database_name] > [filename].sql

数据库恢复命令：mysql -u [username] -p[password] [database_name] < [filename].sql

表授权命令：GRANT [privileges] ON [database_name].[table_name] TO '[username]'@'[host]';

数据更新命令：UPDATE [table_name] SET [column_name] = [value];

2.6 专门说明

本系统涉及到的数据库分为：

proposal_service: 主要存放与“三重一大”决策中的提案相关的业务数据，包括提案信息、审批流程和相关文件等。

user_service：存放系统的用户管理相关业务数据，包括用户信息、权限等。

news-service：存放新闻展示相关的业务数据，包括轮播图信息和新闻内容等。

meeting_service：存放与“三重一大”决策中的会议管理相关的业务数据，包括会议信息、参会人员、会议文件和会议总结等。

开发和测试均在测试服务器上完成。项目验收后，将数据库迁移到集群服务器上。

3 结构设计

在概念结构设计和逻辑结构设计部分仅需描述与新增表、修订表有关的内容，可以引用未做修改的表，但不进行详细描述，系统完整的数据库逻辑结构做为附件附在该文档之后。数据库逻辑结构字典格式参见附件1。

3.1 概念结构设计

1. 新闻服务数据库

设计考虑：该数据库主要负责前端新闻和轮播图的管理与展示。设计时考虑了内容的可配置性、显示状态控制以及排序功能，以便管理员灵活地更新和管理前端信息。

1.1 carousel_image 表 - 轮播图管理

- **实体：**轮播图。
- **定义：**用于存储前端首页轮播图的相关信息。
- **数据项（属性）：**
    - carousel_id（主键ID）：整型（int），非空，自增。唯一标识一张轮播图。值域：正整数。
    - title（轮播图标题）：变长字符串（varchar(100)），非空。轮播图的标题，用于描述图片内容。无计量单位。
    - link_url（点击跳转链接）：变长字符串（varchar(255)），可空。用户点击轮播图后跳转的URL地址。无计量单位。
    - sort_order（排序）：整型（int），非空，默认值 0。数字越小，轮播图在前端显示越靠前。值域：整数。
    - status（状态）：小整型（tinyint），非空，默认值 1。表示轮播图的启用状态。值域：0（停用），1（启用）。

- created_time（创建时间）：日期时间（datetime），非空，默认当前时间戳。记录轮播图创建的时间。计量单位：日期时间。
- updated_time（最后更新时间）：日期时间（datetime），非空，默认当前时间戳并在更新时自动更新。记录轮播图信息最后更新的时间。计量单位：日期时间。
- description（图片描述）：变长字符串（varchar(255)），非空。对轮播图片的简要描述。无计量单位。
- file_path（文件路径）：变长字符串（varchar(255)），可空。存储图片文件在文件系统或云存储中的路径，用于生成访问URL。无计量单位。
- **记录：**每一行代表一个轮播图的完整信息。
- **数据表标识符：**carousel_id。

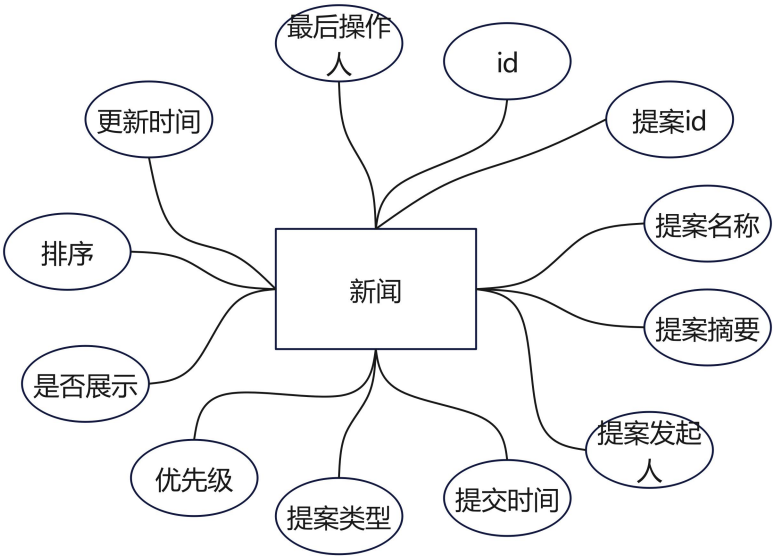

图1 新闻实体E-R图

1.2 news 表 - 新闻展示

- **实体：**新闻。
- **定义：**用于存储前端展示的新闻或公告信息，通常与提案相关联。
- **数据项（属性）：**
  - news_id（表主键，自增）：整型（int），非空，自增。唯一标识一条新闻。值域：正整数。
  - proposal_id（原提案 ID）：整型（int），非空。关联到原始提案的ID，是外键。值域：正整数。

- proposal_name (提案标题): 变长字符串 (varchar(50)), 非空。新闻对应的提案标题。无计量单位。
- proposal_abstract (提案摘要): 文本 (text), 可空。新闻对应的提案的摘要或简介。无计量单位。
- proposal_owner (提案发起人): 变长字符串 (varchar(50)), 非空。发布该新闻所基于的提案的发起人。无计量单位。
- proposal_start_time (提交时间): 变长字符串 (varchar(255)), 非空。新闻所基于的提案的提交时间。格式: YYYY-MM-DD HH。计量单位: 日期时间字符串。
- proposal_type (提案类型): 小整型 (tinyint), 非空, 默认值 0。表示新闻所基于的提案类型。值域: 0 (其他), 1 (重大事项决策) 等。
- proposal_priority (优先级): 小整型 (tinyint), 非空, 默认值 2。表示新闻所基于的提案的优先级。值域: 0 (紧急), 1 (较高), 2 (普通)。
- display_status (是否前端展示): 小整型 (tinyint(1)), 非空, 默认值 1。控制新闻是否在前端显示。值域: 0 (停用), 1 (启用)。
- sort_order (前端自定义排序): 整型 (int), 非空, 默认值 0。数字越小，新闻在前端显示越靠前。值域: 整数。
- created_at (记录创建时间): 日期时间 (datetime), 非空, 默认当前时间戳。记录新闻创建的时间。计量单位: 日期时间。
- updated_at (最后更新时间): 日期时间 (datetime), 非空, 默认当前时间戳并在更新时自动更新。记录新闻信息最后更新的时间。计量单位: 日期时间。
- updated_by (最后操作人): 变长字符串 (varchar(50)), 可空。最后修改该新闻的管理员账号。无计量单位。

- **记录:** 每一行代表一个新闻的完整信息。
- **数据表标识符:** news_id。
- **关系:** news.proposal_id 理论上是 proposal_info.proposal_id 的外键，表示新闻是基于某个提案发布的。

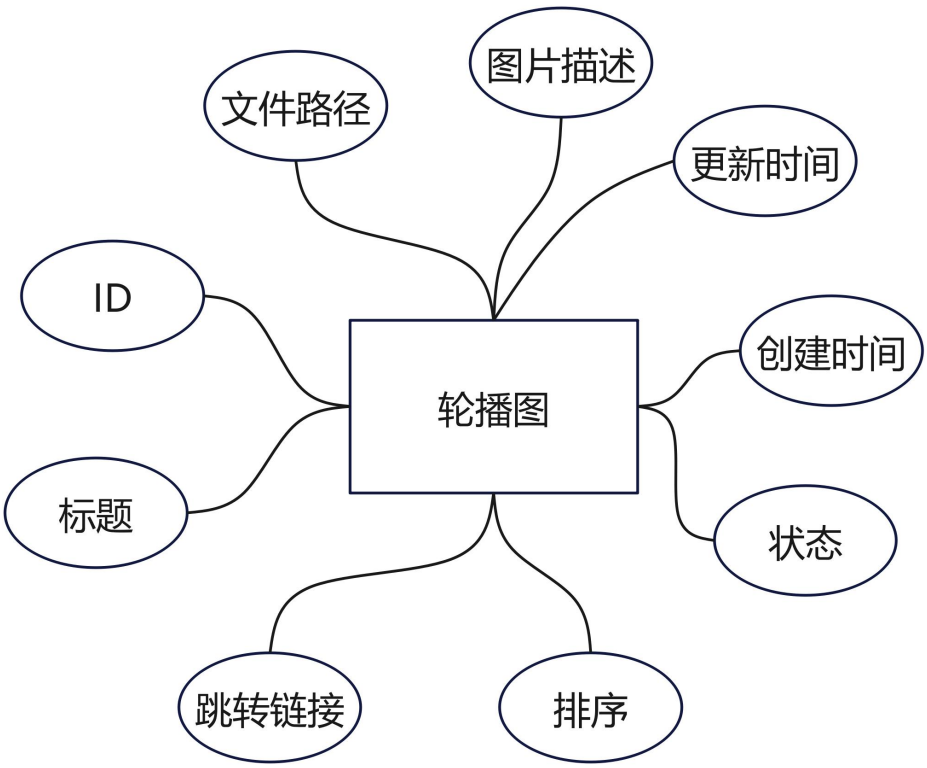

图2 轮播图实体E-R图

2. 会议服务数据库

设计考虑：该数据库用于管理会议的各项信息，包括会议详情、参会人员、会议文件和会议纪要。设计时考虑了会议的生命周期（待开始、进行中、完成）、与提案的关联以及文件和总结的附件功能。

2.1 meeting_conferee 表 - 参会人员

- **实体：**参会人员。
- **定义：**记录了哪些用户参加了具体的会议。
- **数据项（属性）：**
  - conferee_id（主键）：整型（int），非空，自增。唯一标识一条参会记录。值域：正整数。
  - meeting_id（关联会议id）：整型（int），可空。关联到 meeting_info 表的 meetint_id。计量单位：无。
  - user_id（参会人员id）：变长字符串（varchar(255)），可空。参会人员的用户ID，关联到 user_info 表的 id。计量单位：无。
- **记录：**每一行代表一个用户参加一个会议的记录。

- **数据表标识符：** conferee_id。
- **关系：** meeting_conferee.meeting_id 是 meeting_info.meetint_id 的外键；meeting_conferee.user_id 是 user_info.id 的外键。

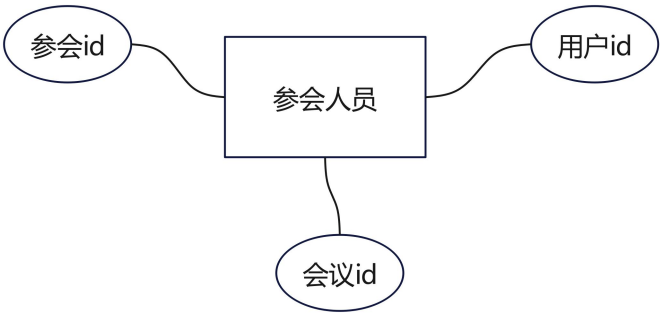

图3 参会人员实体E-R图

2.2 meeting_file 表 - 会议文件

- **实体：** 会议文件。
- **定义：** 存储会议相关的附件，如会议材料或总结材料。
- **数据项（属性）：**
  - file_id（主键）：整型（int），非空，自增。唯一标识一个会议文件。值域：正整数。
  - file_name（文件名称）：变长字符串（varchar(255)），可空。文件的名称。计量单位：无。
  - meeting_id（关联的会议id）：整型（int），可空。关联到 meeting_info 表的 meetint_id。计量单位：无。
  - file_type（文件类型）：小整型（tinyint），可空。表示文件的类型。值域：0（开会上传材料），1（会议完成总结材料）。
- **记录：** 每一行代表一个会议文件。
- **数据表标识符：** file_id。
- **关系：** meeting_file.meeting_id 是 meeting_info.meetint_id 的外键。

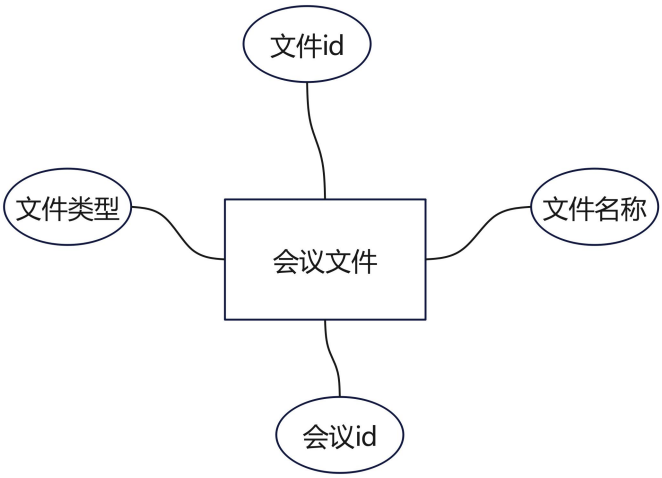

图4 会议文件实体E-R图

2.3 meeting_info 表 - 会议信息

- **实体**：会议。
- **定义**：存储会议的详细信息。
- **数据项（属性）**：
  - meetint_id（主键）：整型（int），非空，自增。唯一标识一个会议。值域：正整数。
  - meeting_name（会议名称）：变长字符串（varchar(255)），可空。会议的名称或主题。计量单位：无。
  - meeting_owner（会议发起人）：变长字符串（varchar(100)），可空。发起该会议的用户ID，只有发起人才能结束会议。关联到 user_info.id。计量单位：无。
  - metting_department（会议发起人部门）：小整型（tinyint），可空。会议发起人所属的部门，与用户表的部门定义相同。值域：0-9（参照 user_info.department 定义）。
  - meeting_content（会议内容）：文本（text），可空。会议的具体议程、讨论内容等。计量单位：无。
  - meeting_start_time（会议开始时间）：变长字符串（varchar(255)），可空。会议的实际开始时间。计量单位：日期时间字符串。
  - meeting_end_time（会议自动结束时间）：变长字符串（varchar(255)），可空。会议的预计结束时间，可能用于自动结束会议。计量单位：日期时间字符串。

- proposal_id (提案ID): 整型 (int), 可空。与该会议关联的提案ID，关联到 proposal_info 表的 proposal_id。计量单位: 无。
- meeting_status (会议状态): 小整型 (tinyint), 可空。表示会议的当前状态。值域: 0 (待开始), 1 (进行中), 2 (完会)。
- meeting_type (会议类型): 小整型 (tinyint), 可空。表示会议的性质或分类。值域: 0 (其他), 1 (重大事项决策), 2 (重要干部任免), 3 (重大投资项目决策), 4 (大额资金使用)。

- **记录:** 每一行代表一个会议的完整信息。
- **数据表标识符:** meetint_id。
- **关系:** meeting_info.meeting_owner 理论上是 user_info.id 的外键；meeting_info.proposal_id 是 proposal_info.proposal_id 的外键。

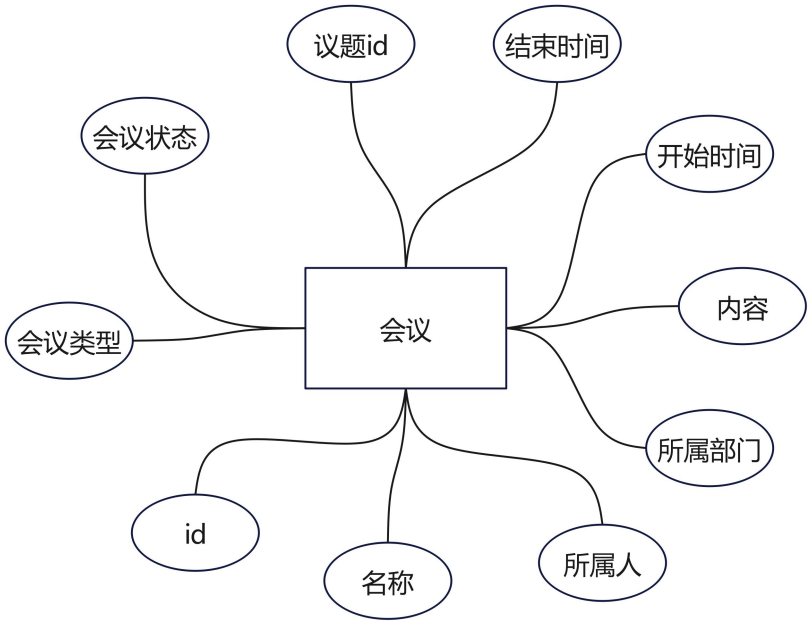

图5 会议实体E-R图

2.4 meeting_summary 表 - 会议总结

- **实体:** 会议总结。
- **定义:** 存储会议结束后生成的总结内容，可能由AI生成。
- **数据项 (属性):**
  - summary_id (主键): 整型 (int), 非空, 自增。唯一标识一条会议总结。值域: 正整数。

- meeting_id (关联的会议id): 整型 (int), 可空。关联到 meeting_info 表的 meetint_id。计量单位: 无。
- approval_id (审批ID): 整型 (int), 可空。可能关联到某个审批流程的 ID。计量单位: 无。
- summary_content (AI总结生成): 文本 (text), 可空。会议的总结内容，可能由AI工具自动生成。计量单位: 无。

- **记录:** 每一行代表一个会议的总结。
- **数据表标识符:** summary_id。
- **关系:** meeting_summary.meeting_id 是 meeting_info.meetint_id 的外键; meeting_summary.approval_id 可能关联到 proposal_approval.approval_id。

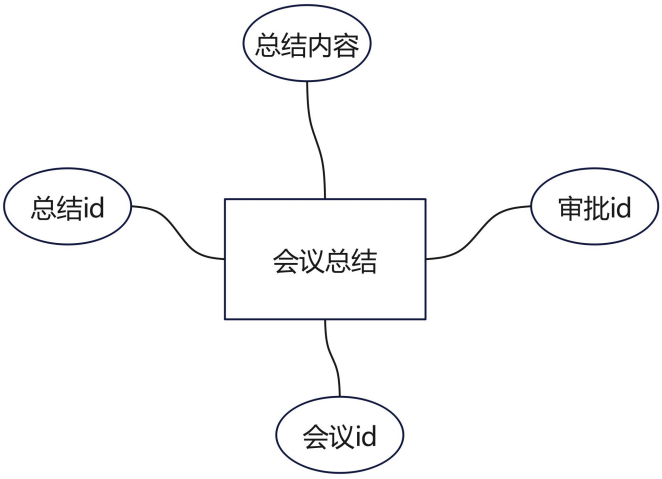

图6 会议总结实体E-R图

3. 提案服务数据库

设计考虑: 该数据库负责管理提案的整个生命周期，包括提案的提交、审批、文件附件以及最终的评分和状态。设计时强调了审批流程的层级、提案的优先级以及文件管理。

3.1 proposal_approval 表 - 提案审批记录

- **实体:** 提案审批记录。
- **定义:** 记录提案在不同部门或层级的审批情况。
- **数据项 (属性):**
  - approval_id (主键): 整型 (int), 非空, 自增。唯一标识一条审批记录。值域: 正整数。

- proposal_id（提案id）：整型（int），可空。关联到 proposal_info 表的 proposal_id。计量单位：无。
- approval_level（审批级别）：小整型（tinyint），可空。表示提案审批到了第几级，等同于 proposal_info.proposal_level。计量单位：级别。
- approval_department（审批部门）：小整型（tinyint），可空。执行审批的部门。值域：0-9（参照 user_info.department 定义）。
- approval_status（审批状态）：小整型（tinyint），可空。表示该级审批的结果。值域：0（未审批），1（同意），2（拒绝）。
- approval_time（审批时间）：变长字符串（varchar(255)），可空。该级审批完成的时间。格式：yyyy-mm-dd hh。计量单位：日期时间字符串。
- approval_opinion（审批意见）：文本（text），可空。审批人给出的意见或批注。计量单位：无。
- approval_user（审批人id）：变长字符串（varchar(100)），可空。执行审批操作的用户ID，关联到 user_info.id。计量单位：无。

- **记录：**每一行代表提案在某个级别或部门的一次审批记录。
- **数据表标识符：**approval_id。
- **关系：**proposal_approval.proposal_id 是 proposal_info.proposal_id 的外键；proposal_approval.approval_user 是 user_info.id 的外键。

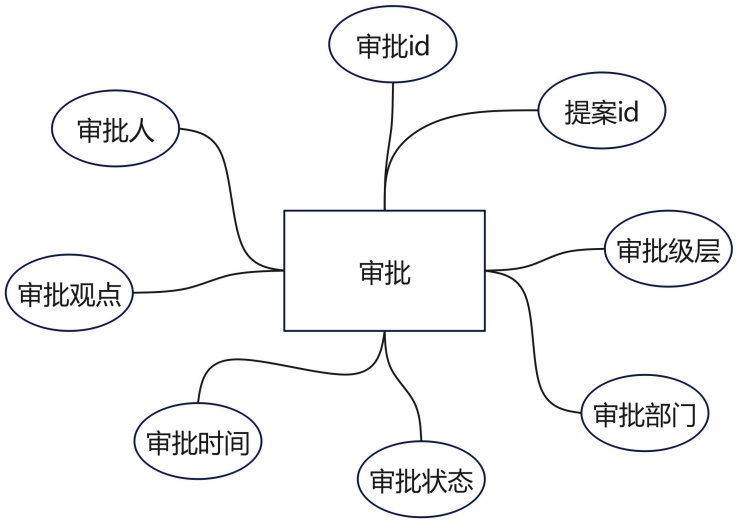

图7 审批实体E-R图

3.2 proposal_file 表 - 提案文件

- **实体**：提案文件。
- **定义**：存储与提案相关的附件。
- **数据项（属性）**：
  - file_id（主键）：整型（int），非空，自增。唯一标识一个提案文件。值域：正整数。
  - file_name（文件名称）：变长字符串（varchar(255)），可空。文件的名称。计量单位：无。
  - proposal_id（提案的id）：整型（int），可空。关联到 proposal_info 表的 proposal_id。计量单位：无。
- **记录**：每一行代表一个提案附件。
- **数据表标识符**：file_id。
- **关系**：proposal_file.proposal_id 是 proposal_info.proposal_id 的外键。

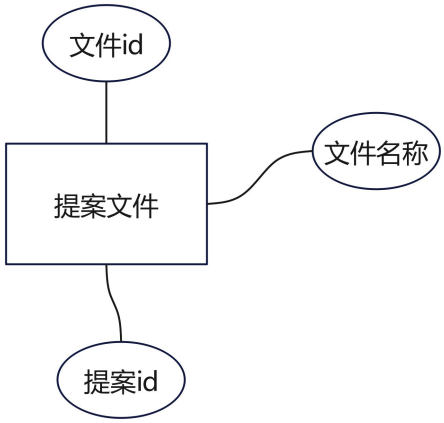

图8 提案文件实体E-R图

3.3 proposal_info 表 - 提案信息

- **实体**：提案。
- **定义**：存储提案的详细信息和当前状态。
- **数据项（属性）**：
  - proposal_id（主键）：整型（int），非空，自增。唯一标识一个提案。值域：正整数。

- proposal_type (提案类型): 小整型 (tinyint), 可空。表示提案的性质或分类。值域: 0 (其他), 1 (重大事项决策), 2 (重要干部任免), 3 (重大投资项目决策), 4 (大额资金使用)。
- proposal_name (提案名称): 变长字符串 (varchar(50)), 可空。提案的标题。计量单位: 无。
- proposal_owner (提案发起人): 变长字符串 (varchar(50)), 可空。提交提案的用户ID, 关联到 user_info.id。计量单位: 无。
- proposal_abstract (提案摘要): 文本 (text), 可空。提案内容的摘要或简介。计量单位: 无。
- proposal_start_time (提案开始时间): 变长字符串 (varchar(255)), 可空。提案提交的时间。格式: YYYY-MM-DD HH。计量单位: 日期时间字符串。
- proposal_level (议题审批到第几级了): 小整型 (tinyint), 可空。表示提案当前审批进行到的级别。计量单位: 级别。
- proposal_priority (优先级): 小整型 (tinyint), 可空。表示提案的紧急程度。值域: 0 (紧急), 1 (较高), 2 (普通)。
- proposal_rate (平均评分): 双精度浮点数 (double), 可空。提案完成后的平均评分。值域: 0.0-5.0（假设）。
- proposal_status (提案状态): 小整型 (tinyint(1)), 可空。表示提案的整体审批状态。值域: 0 (审批中), 1 (审批完成)。
- proposal_end_time (提案结束时间): 二进制变长字符串 (varbinary(255)), 可空。提案完成或结束的时间。计量单位: 日期时间字符串。

- **记录:** 每一行代表一个提案的完整信息。
- **数据表标识符:** proposal_id。
- **关系:** proposal_info.proposal_owner 是 user_info.id 的外键。

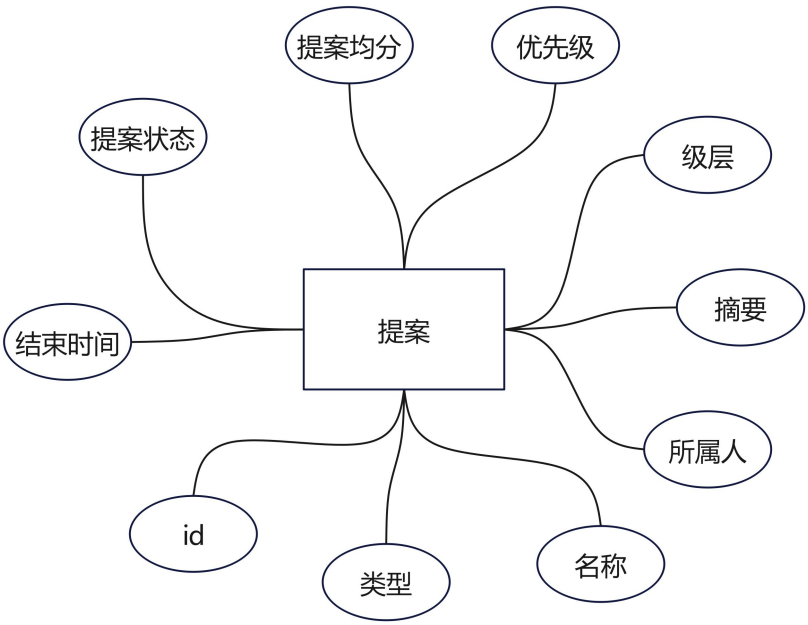

图9 提案实体E-R图

4. 用户服务数据库

设计考虑：该数据库主要负责用户信息的管理，包括登录凭证、基本信息、权限和所属部门。设计时考虑了用户的身份识别、权限控制以及组织结构（部门）的划分。

4.1 user_info 表 - 用户信息

- **实体：**用户。
- **定义：**存储系统用户的基本信息、登录凭证和权限。
- **数据项（属性）：**
  - id（主键 登录）：变长字符串（varchar(20)），非空。用户的唯一标识符，也作为登录账号。无计量单位。
  - password（密码）：变长字符串（varchar(30)），可空。用户的登录密码。无计量单位。
  - name（姓名）：变长字符串（varchar(20)），可空。用户的真实姓名。无计量单位。
  - is_admin（是否管理员）：小整型（tinyint(1)），可空。表示用户是否为管理员。值域：0（非管理员），1（管理员）。
  - department（部门）：小整型（tinyint），可空。用户所属的部门。值域：0（系统管理员），1（校长办公室），2（教务处），3（财政部），4（学工），5（人事部），6（石油学院），7（工学院），8（文理学院），9（工商马院）。

- phone（电话）：变长字符串（varchar(255)），可空。用户的联系电话。计量单位：无。
- email（邮箱）：变长字符串（varchar(255)），可空。用户的电子邮箱。计量单位：无。
- office（办公室）：变长字符串（varchar(255)），可空。用户的办公室位置。计量单位：无。

- **记录：**每一行代表一个用户的完整信息。
- **数据表标识符：**id。

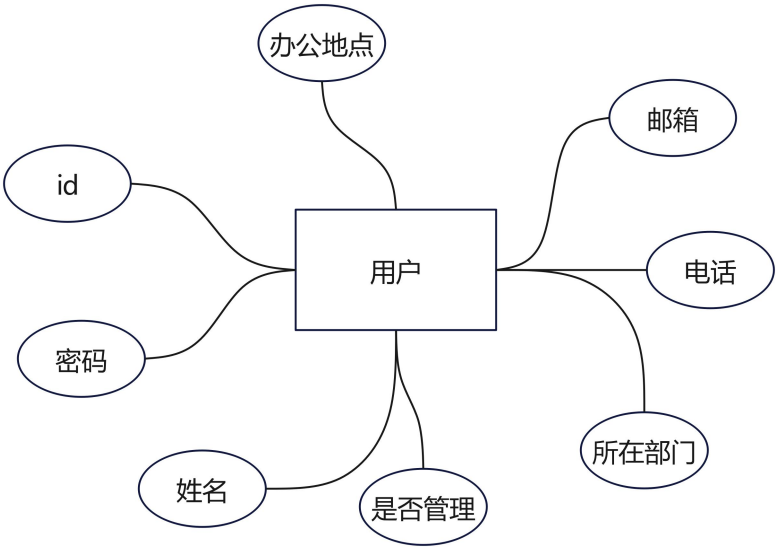

图10 用户实体E-R图

实体关系图（E-R Diagram）概述

基于上述详细说明，可以绘制出以下主要实体之间的关系：

- user_info（用户）
  - user_info 1:N meeting_conferee（一个用户可以参与多个会议）
  - user_info 1:N meeting_info（一个用户可以发起多个会议，作为meeting_owner）
  - user_info 1:N proposal_info（一个用户可以发起多个提案，作为proposal_owner）
  - user_info 1:N proposal_approval（一个用户可以审批多个提案，作为approval_user）

- user_info 1:N news（一个用户可能是新闻相关提案的发起人 proposal_owner）

- **proposal_info（提案）**
  - proposal_info 1:N proposal_approval（一个提案有多个审批记录，对应不同的审批级别或部门）
  - proposal_info 1:N proposal_file（一个提案可以有多个附件文件）
  - proposal_info 1:1 news（一个提案可能对应一条新闻，news.proposal_id 是唯一的）
  - proposal_info 1:N meeting_info（一个提案可以关联到一个或多个会议）
- **meeting_info（会议）**
  - meeting_info 1:N meeting_conferee（一个会议可以有多个参会人员）
  - meeting_info 1:N meeting_file（一个会议可以有多个附件文件，包括会议前材料和会议总结材料）
  - meeting_info 1:1 meeting_summary（一个会议通常对应一个总结）
- **carousel_image（轮播图）**
  - 独立实体，主要用于前端展示管理，不直接与其他业务实体有直接关联。

3.2 逻辑结构设计

详细说明本数据库的数据库管理员视图，即把上述原始数据进行分解、合并后重新组织起来的数据库全局逻辑结构，包括所确定的关键字和属性、重新确定的记录结构和数据表结构、所建立的各个数据表之间的相互关系，并参照新疆油田公司《勘探开发数据库数据表编码规范（Q/SY XJ0204-2009）》以及《数据库逻辑结构管理规范（Q/SY XJ0205-2009）》等相关标准设计《数据库逻辑结构》。并绘制E_R图，要求达到第三范式。

全局ER图如下所示：

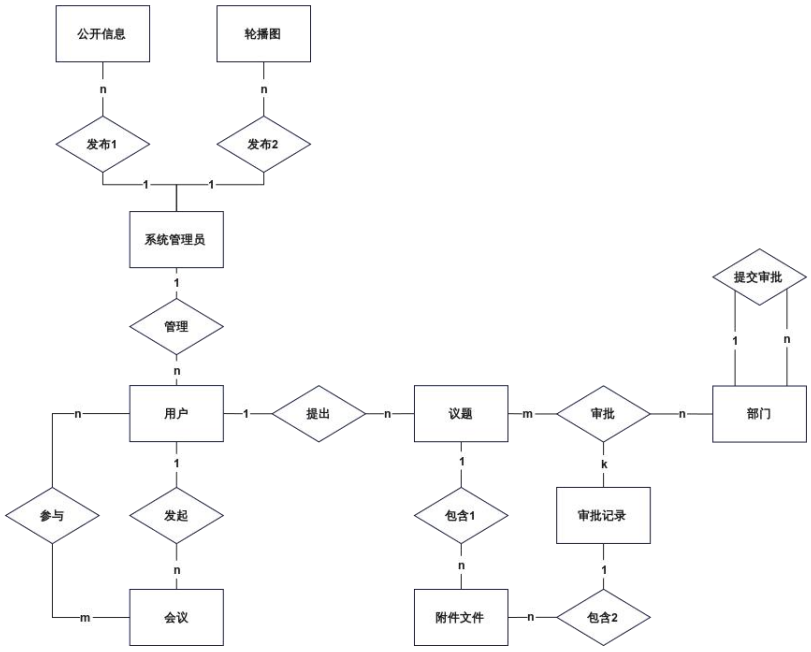

图11 整体E-R图

1. 实体与关键属性识别

- 用户（User）
  - 关键字：user_id
  - 属性：password，name，is_admin，department_id（新外键），phone，email，office
- 部门（Department）
  - 关键字：department_id
  - 属性：department_name
- 提案（Proposal）
  - 关键字：proposal_id
  - 属性：proposal_type，proposal_name，proposal_owner（外键：user_id），proposal_abstract，proposal_start_time，proposal_level，proposal_priority，proposal_rate，proposal_status，proposal_end_time
- 提案审批（ProposalApproval）
  - 关键字：approval_id

- 属性：proposal_id（外键：proposal_id），approval_level，approval_department_id（新外键），approval_status，approval_time，approval_opinion，approval_user（外键：user_id）

- **提案文件（ProposalFile）**
  - **关键字：**file_id
  - **属性：**file_name，proposal_id（外键：proposal_id）
- **会议（Meeting）**
  - **关键字：**meeting_id（原 meeting_info.meetint_id）
  - 属性：meeting_name，meeting_owner（外键：user_id），meeting_department_id，meeting_content，meeting_start_time，meeting_end_time，proposal_id（外键：proposal_id），meeting_status，meeting_type
- **参会人员（MeetingConferee）**
  - **关键字：**conferee_id
  - **属性：**meeting_id（外键：meeting_id），user_id（外键：user_id）
- **会议文件（MeetingFile）**
  - **关键字：**file_id
  - **属性：**file_name，meeting_id（外键：meeting_id），file_type
- **会议总结（MeetingSummary）**
  - **关键字：**summary_id
  - 属性：meeting_id（外键：meeting_id），approval_id（外键：proposal_approval.approval_id，潜在关联），summary_content
- **新闻（News）**
  - **关键字：**news_id
  - **属性：**proposal_id（外键：proposal_id），display_status，sort_order，created_at，updated_at，updated_by（外键：user_id）
- **轮播图（CarouselImage）**
  - **关键字：**carousel_id
  - 属性：title，link_url，sort_order，status，created_time，updated_time，description，file_path

2.记录结构和数据表结构

2.1 用户管理模块

- User 表
  - **记录结构:**(user_id, password, name, is_admin, department_id, phone, email, office)
  - **主键:** user_id
  - **外键:** department_id 引用 Department.department_id (新增外键)
  - **原表名:** user_info
  - **字段对应:**
    - id -> user_id
    - department -> department_id
- Department 表
  - **记录结构:** (department_id, department_name)
  - **主键:** department_id
  - **设计考虑 (3NF):** 独立存储部门信息，消除 User 表中的传递依赖。
  - **数据项:**
    - department_id: 小整型 (tinyint), 主键。值域: 0 (系统管理员), 1 (校长办公室), 2 (教务处) ... 9 (工商马院)。
    - department_name: 变长字符串 (varchar), 非空。部门的中文名称。

2.2 提案管理模块

- Proposal 表
  - **记录结构:** (proposal_id, proposal_type, proposal_name, proposal_owner, proposal_abstract, proposal_start_time, proposal_level, proposal_priority, proposal_rate, proposal_status, proposal_end_time)
  - **主键:** proposal_id
  - **外键:** proposal_owner 引用 User.user_id
  - **设计考虑 (3NF):** 所有非主属性完全依赖于 proposal_id 且不传递依赖。
- ProposalApproval 表

- **记录结构**: (approval_id, proposal_id, approval_level, approval_department_id, approval_status, approval_time, approval_opinion, approval_user)
  - **主键**: approval_id
  - **外键**: proposal_id 引用 Proposal.proposal_id, approval_department_id 引用 Department.department_id, approval_user 引用 User.user_id

- **ProposalFile 表**
  - **记录结构**: (file_id, file_name, proposal_id)
  - **主键**: file_id
  - **外键**: proposal_id 引用 Proposal.proposal_id

2.3 会议管理模块

- **Meeting 表**
  - **记录结构**: (meeting_id, meeting_name, meeting_owner, meeting_department_id, meeting_content, meeting_start_time, meeting_end_time, proposal_id, meeting_status, meeting_type)
  - **主键**: meeting_id
  - **外键**: meeting_owner 引用 User.user_id, meeting_department_id 引用 Department.department_id, proposal_id 引用 Proposal.proposal_id
  - **原表名**: meeting_info
  - **字段对应**:
    - meetint_id -> meeting_id
    - metting_department -> meeting_department_id

- **MeetingConferee 表** (对应原 meeting_conferee 表)
  - **记录结构**: (conferee_id, meeting_id, user_id)
  - **主键**: conferee_id
  - **外键**: meeting_id 引用 Meeting.meeting_id, user_id 引用 User.user_id
  - **设计考虑 (3NF)**: 所有非主属性完全依赖于 conferee_id 且不传递依赖。

- MeetingFile 表（对应原 meeting_file 表）
  - **记录结构**：(file_id, file_name, meeting_id, file_type)
  - **主键**：file_id
  - **外键**：meeting_id 引用 Meeting.meeting_id
  - **设计考虑（3NF）**：所有非主属性完全依赖于 file_id 且不传递依赖。
- MeetingSummary 表
  - **记录结构**：(summary_id, meeting_id, approval_id, summary_content)
  - **主键**：summary_id
  - **外键**：meeting_id 引用 Meeting.meeting_id, approval_id 引用 ProposalApproval.approval_id（潜在关联，需业务逻辑确认）
  - **设计考虑（3NF）**：所有非主属性完全依赖于 summary_id 且不传递依赖。

2.4 新闻与前端展示模块

- News 表
  - **记录结构**：(news_id, proposal_id, display_status, sort_order, created_at, updated_at, updated_by)
  - **主键**：news_id
  - **外键**：proposal_id 引用 Proposal.proposal_id, updated_by 引用 User.user_id
  - **设计考虑（3NF）**：**关键改进**。移除了 proposal_name, proposal_abstract, proposal_owner, proposal_start_time, proposal_type, proposal_priority 等字段。这些字段通过 proposal_id 可以在 Proposal 表中获取。这样就消除了 news_id -> proposal_id -> 冗余提案信息 的传递依赖，使 News 表符合3NF。
- CarouselImage 表
  - **记录结构**：(carousel_id, title, link_url, sort_order, status, created_time, updated_time, description, file_path)
  - **主键**：carousel_id
  - **设计考虑（3NF）**：所有非主属性完全依赖于 carousel_id 且不传递依赖。

3. 所建立的各个数据表之间的相互关系

以下是数据库中主要实体之间的关系，使用"一对一"(1:1)、"一对多"(1:N) 和 "多对多"(M:N) 表示：

- **User（用户）表:**
  - 与 Department（部门）表: **N:1**（多个用户属于一个部门）
    - User.department_id -> Department.department_id（新增关系）
  - 与 Proposal（提案）表: **1:N**（一个用户可以发起多个提案）
    - Proposal.proposal_owner -> User.user_id
  - 与 ProposalApproval（提案审批）表: **1:N**（一个用户可以进行多次审批）
    - ProposalApproval.approval_user -> User.user_id
  - 与 Meeting（会议）表: **1:N**（一个用户可以发起多个会议）
    - Meeting.meeting_owner -> User.user_id
  - 与 MeetingConferee（参会人员）表: **1:N**（一个用户可以参与多个会议）
    - MeetingConferee.user_id -> User.user_id
  - 与 News（新闻）表: **1:N**（一个用户可以作为新闻的更新者）
    - News.updated_by -> User.user_id
- **Department（部门）表:**
  - 与 User（用户）表: **1:N**（一个部门包含多个用户）
  - 与 ProposalApproval（提案审批）表: **1:N**（一个部门可以进行多次审批）
    - ProposalApproval.approval_department_id -> Department.department_id（新增关系）
  - 与 Meeting（会议）表: **1:N**（一个部门可以发起多个会议）
    - Meeting.meeting_department_id -> Department.department_id（新增关系）
- **Proposal（提案）表:**
  - 与 ProposalApproval（提案审批）表: **1:N**（一个提案有多个审批记录）
    - ProposalApproval.proposal_id -> Proposal.proposal_id
  - 与 ProposalFile（提案文件）表: **1:N**（一个提案可以有多个附件文件）
    - ProposalFile.proposal_id -> Proposal.proposal_id

- 与 Meeting（会议）表：1:N（一个提案可以关联到多个会议）
  - Meeting.proposal_id -> Proposal.proposal_id
- 与 News（新闻）表：1:1（一个提案通常只对应一条新闻，通过 news.proposal_id 上的唯一索引强制）
  - News.proposal_id -> Proposal.proposal_id

- **Meeting（会议）表：**
  - 与 MeetingConferee（参会人员）表：1:N（一个会议可以有多个参会人员）
    - MeetingConferee.meeting_id -> Meeting.meeting_id
  - 与 MeetingFile（会议文件）表：1:N（一个会议可以有多个附件文件）
    - MeetingFile.meeting_id -> Meeting.meeting_id
  - 与 MeetingSummary（会议总结）表：1:1（一个会议通常对应一个总结）
    - MeetingSummary.meeting_id -> Meeting.meeting_id
- **MeetingConferee（参会人员）表：**
  - 这是一个连接表，实现了 User 和 Meeting 之间的多对多关系。
- **CarouselImage（轮播图）表：**
  - 独立实体，主要用于前端展示管理，不直接与其他业务实体建立外键关系。

3.3 物理结构设计

详细说明本数据库的系统程序员视图，即数据在内存中的安排，包括对索引区、缓冲区的设计；所使用的外存设备及外存空间的组织，包括索引区、数据块的组织与划分以及访问数据的方式方法。

3.3.1 设计数据的存取路径

在创建索引的时候，一般遵循以下的一些经验性原则：

1) 在经常需要搜索的列上建立索引。

2) 在主关键字上建立索引。

3) 在经常用于连接的列上建立索引，即在外键上建立索引。

4) 在经常需要根据范围进行搜索的列上创建索引，因为索引已经排序，其指定的范围是连续的。

5) 在经常需要排序的列上建立索引，因为索引已经排序，这样查询可以利用索引的排序，加快排序查询的时间。

6) 在经常成为查询条件的列上建立索引。也就是说，在经常使用在WHERE子句中的列上面建立索引。

7) 对于那些在查询中很少使用和参考的列不应该创建索引。因为既然这些列很少使用到，有索引并不能提高查询的速度。相反，由于增加了索引，反而降低了系统的维护速度和增大了空间需求。

8) 对于那些只有很少值的列不应该建立索引。

9) 过长的属性不应该建立索引，例如超过30个字节。因为在过长的属性上建立索引，索引所占的存储空间较大，而索引的级数也随之增加，有诸多不利之处。如果实在需要在其上建立索引，必须采取索引属性压缩的措施。

10) 经常更新的属性或表不应该建立索引。因为在更新时有关的索引需要做相应的修改。

3.3.2 设计数据的存放位置

1) 数据库数据备份、日志文件备份等由于只在故障恢复时才使用，而且数据量很大，可以存放在磁带上。

2) 将表和索引分别放在不同的磁盘上。

3) 将比较大的表分别放在两个磁盘上，以加快存取速度。

4) 将数据分布到多个磁盘上。

4、其它设计

4.1 完整性设计

数据完整性是数据库管理系统中的一个核心概念，它确保了数据的准确性、一致性和可靠性。本项目通过以下设计来维护数据完整性：

- **实体完整性** (Entity Integrity):

- **主键 (Primary Key)**：每个表都定义了主键，确保每条记录的唯一性，并且主键值不能为空 (NOT NULL)。例如，user_info 表的 id 字段、proposal_info 表的 proposal_id 字段、meeting_info 表的 meetint_id 字段、news 表的 news_id 字段 和 carousel_image 表的 carousel_id 字段 都被定义为主键，且设置为 AUTO_INCREMENT 或 NOT NULL，确保了每条记录在各自表中的唯一标识。
- **唯一索引 (Unique Index)**：除了主键，某些字段也创建了唯一索引以保证特定属性的唯一性。例如，news 表在 proposal_id 上创建了 uk_news_proposal 唯一索引，确保一个提案只对应一条新闻，避免了新闻数据的重复。

- **参照完整性 (Referential Integrity)**：
  - **外键 (Foreign Key)**：数据库中通过外键约束来维护表与表之间的关系，确保关联数据的一致性。虽然提供的SQL脚本中没有显式定义FOREIGN KEY约束语法（如CONSTRAINT ... FOREIGN KEY ... REFERENCES ...），但通过注释和字段命名，可以推断出存在以下逻辑上的外键关系：
    - news.proposal_id 引用 proposal_info.proposal_id。
    - proposal_approval.proposal_id 引用 proposal_info.proposal_id。
    - proposal_approval.approval_user 引用 user_info.id。
    - proposal_file.proposal_id 引用 proposal_info.proposal_id。
    - meeting_conferee.meeting_id 引用 meeting_info.meetint_id。
    - meeting_conferee.user_id 引用 user_info.id。
    - meeting_file.meeting_id 引用 meeting_info.meetint_id。
    - meeting_info.proposal_id 引用 proposal_info.proposal_id。
    - meeting_info.meeting_owner 引用 user_info.id。
    - meeting_summary.meeting_id 引用 meeting_info.meetint_id。
    - meeting_summary.approval_id 可能引用 proposal_approval.approval_id。
  - 在实际部署中，应该显式添加 FOREIGN KEY 约束，并配置适当的参照动作（如 ON DELETE RESTRICT 或 ON DELETE CASCADE），以防止删除父表记录时留下孤儿记录，或在更新父表主键时同步更新子表外键。

- **域完整性 (Domain Integrity)**：

- **数据类型 (Data Types)**: 为每个属性选择合适的数据类型，限制其取值范围和格式。例如，使用 int 用于ID、varchar 用于字符串、tinyint 用于状态或类型标识、datetime 用于时间戳、text 用于长文本。
- **NULL 约束 (NULL Constraint)**: 根据业务需求，对重要的、必须有值的字段设置为 NOT NULL，确保数据的完整性。例如，carousel_image 表的 title、sort_order、status、created_time、updated_time 和 description 字段均设置为 NOT NULL。news 表的关键字段如 proposal_id、proposal_name、proposal_owner、proposal_start_time、proposal_type、proposal_priority、display_status、sort_order、created_at 和 updated_at 也都是 NOT NULL。
- **默认值 (Default Values)**: 为某些字段设置默认值，以确保在插入新记录时这些字段不会为空或拥有预期值。例如，carousel_image 表的 sort_order 默认值为 0，status 默认值为 1。news 表的 proposal_type 默认值为 0，proposal_priority 默认值为 2，display_status 默认值为 1。

- **用户定义完整性 (User-Defined Integrity)**:
  - **业务逻辑约束**: 数据库层面不易强制的业务规则，通过应用程序逻辑进行验证。例如，proposal_rate（平均评分）的值域（如0-5分）需要在应用层进行验证和计算。

2. 数据库后援 (Backup) 频率

数据库后援是灾难恢复和数据完整性保障的关键环节。虽然SQL文件本身不包含备份策略，但基于其业务重要性，可以推断出以下考虑：

- **全量备份**:
  - **频率**: 建议每周进行一次全量备份，例如在业务低峰期（如周末凌晨）。
  - **目的**: 捕获数据库在某个时间点的完整状态。
- **增量/差异备份**:
  - **频率**: 每天进行一次增量或差异备份。
  - **目的**: 记录上次全量备份或增量/差异备份以来的数据变化，缩短备份时间。
- **事务日志备份 (对于支持的DB)**:
  - **频率**: 对于生产环境，可能需要每小时甚至更频繁地备份事务日志。

- **目的**：实现Point-in-Time Recovery，将数据库恢复到任意时间点，最大程度减少数据丢失。

- **异地存储**：备份文件应存储在与主数据库服务器物理隔离的位置，以防范机房级灾难。

3. 数据共享考虑

该数据库系统明显设计为支持多服务（新闻服务、提案服务、会议服务、用户服务）的数据共享，主要通过以下方式实现：

- **服务间数据关联**：不同服务的数据表之间通过逻辑外键（例如 proposal_id 在 news、meeting_info 和 proposal_approval 中出现）进行关联，使得各服务可以协同工作，共享核心实体信息。例如，新闻服务可以获取提案的标题和摘要进行展示，会议服务可以关联到具体的提案，审批服务则直接围绕提案进行。

- **统一的用户管理**：user_service 集中管理所有用户数据。其他服务（如提案发起人 proposal_owner、会议发起人 meeting_owner、审批人 approval_user、参会人员 meeting_conferee.user_id）都引用 user_info.id，实现了用户信息的集中管理和共享。

- **API 接口设计**：在实际应用中，不同服务通常会通过定义良好的API接口来共享数据，而不是直接跨服务访问数据库。这有助于解耦服务，提高系统的可伸缩性和可维护性。

4. 数据冗余考虑

在数据库设计中，完全消除冗余通常会增加查询时的复杂性和连接操作，从而影响性能。因此，在某些情况下会允许适度的冗余以优化读取性能，但这需要权衡。

- **可接受的冗余**：
  - **news 表中的冗余信息**：news 表中存储了 proposal_name、proposal_abstract、proposal_owner、proposal_start_time、proposal_type、proposal_priority 等字段。这些信息实际上在 proposal_info 表中都有。这种冗余的设计考虑是为了前端展示新闻时，可以快速获取提案的关键信息，避免了每次查询新闻时都进行 news 和 proposal_info 表的JOIN操作，从而提高查询性能。然而，这也意味着当 proposal_info 中的这些字段更新时，news 表中对应的字段也需要同步更新，否则会导致数据不一致。

- **避免的冗余**：
  - **非规范化数据**：通过遵循范式（如3NF）设计，可以最大程度地减少不必要的冗余。例如，user_info 表中的 department 字段的值是数字代码，这本身是

可接受的，但如果直接存储部门名称而非代码，且部门名称在多个用户记录中重复，则会导致冗余。更好的做法是建立一个独立的 Department 表来管理部门信息，并通过外键关联。原SQL文件中部门信息是数字代码，如果其对应的中文名称存储在代码中，则属于轻微的非规范化，但在实际中为了避免额外的JOIN操作可能被接受。

- **派生数据**: 避免存储可以通过其他数据计算或派生出来的数据，除非是为了性能优化。例如，proposal_info.proposal_rate (平均评分) 如果是通过所有审批记录的评分计算得来，则属于派生数据。存储它可以加快查询，但需要确保在源数据变化时同步更新。

4.2 安全保密设计

说明在数据库的设计中，将如何通过区分不同的访问者、不同的访问类型和不同的数据对象等而获得数据库安全保密的设计考虑。以及将要采用的保证数据安全保密的措施和机制，如数据库安全破坏标识、资源保护方式、存取控制方式等。

4.2.1. 数据库安全保密的设计考虑

4.2.1.1 区分不同的访问者 (Authentication & Authorization)

- **用户身份认证 (Authentication)：**
  - **唯一用户标识**: user_info 表中的 id 字段作为用户登录的主键，保证每个用户的唯一性。这是区分不同访问者的基础。
  - **密码管理**: user_info 表包含 password 字段用于用户登录。在实际应用中，密码应进行加密存储（如哈希加盐），而非明文存储，以防止密码泄露。
  - **管理员标识**: user_info 表中的 is_admin 字段明确区分普通用户和管理员 (0 非管理员，1 管理员)，这是实现基于角色的访问控制的基础。
- **用户角色与权限分配 (Authorization)：**
  - **部门权限**: user_info 表中的 department 字段（以及3NF改造后的 department_id 关联 Department 表），可以作为细粒度权限控制的依据。例如，教务处的用户只能访问教务相关提案和会议信息。
  - **功能角色**: 除了 is_admin，可以根据业务需求定义更细致的功能角色（如“提案发起人”、“会议组织者”、“审批人”），并将用户与这些角色关联。例如，

proposal_owner 字段用于标识提案发起人，meeting_owner 字段用于标识会议发起人，他们拥有修改各自提案或会议的特权。

4.2.1.2 区分不同的访问类型 (Access Control Types)

- **读写分离**: 数据库操作分为读 (SELECT) 和写 (INSERT, UPDATE, DELETE) 操作。
  - **只读权限**: 对于普通用户，可以限制他们只能读取某些公开信息，如 news 表和 carousel_image 表的内容。
  - **写权限**: 只有特定角色（如管理员、提案发起人、会议组织者）才被允许对各自负责的数据进行修改、新增或删除操作。例如，只有 meeting_owner 才能结束会议。
- **操作粒度控制**:
  - **数据行级安全 (Row-Level Security, RLS)**: 允许用户只能访问或修改其“拥有”的数据行。例如，一个用户只能查看自己发起的提案 (proposal_owner = user_id) 或审批过的提案 (approval_user = user_id)。
  - **数据列级安全 (Column-Level Security, CLS)**: 限制用户对特定表中的某些列的访问。例如，普通用户可能无法查看 user_info 表中的 phone 或 email 等敏感信息。

5.2.1.3 区分不同的数据对象 (Data Object Protection)

- **表级安全**: 对整个表设置访问权限。例如，只有管理员可以访问 user_info 表。
- **视图级安全**: 创建视图 (Views) 可以限制用户只能看到部分数据或某些表的聚合数据。例如，可以创建一个只包含新闻公开字段的视图，供前端展示使用，隐藏其内部关联的提案ID等敏感信息。
- **存储过程/函数级安全**: 限制用户直接操作底层表，而是通过存储过程或函数来执行预定义的操作，从而对数据访问进行封装和控制。例如，更新提案状态的操作可以通过一个存储过程来完成，其中包含权限检查逻辑。
- **文件安全**: carousel_image.file_path 、 meeting_file.file_name 和 proposal_file.file_name 存储文件路径。这些文件本身需要存储在安全的服务器上，并配合文件存储服务的权限管理（如生成签名URL），确保只有授权用户才能访问。

4.2.2. 保证数据安全保密的措施和机制

4.2.2.1 数据库安全破坏标识 (Security Event Logging & Monitoring)

- **审计日志 (Audit Trails)**: 记录所有敏感操作，包括：
    - **登录尝试**: 记录所有用户（包括失败）的登录尝试，包括时间、IP地址、用户ID。
    - **数据修改**: 记录对关键数据（如 proposal_info、user_info）的 INSERT, UPDATE, DELETE 操作，包括操作用户、时间、修改前后的值。
    - **权限变更**: 记录对用户权限的修改操作。
    - **created_at / updated_at / updated_time / updated_by 字段**: news 表、carousel_image 表包含 created_at/created_time 和 updated_at/updated_time 字段，用于记录数据创建和最后更新的时间戳。news 表中的 updated_by 字段用于记录最后操作的管理员账号。这些时间戳和操作人信息是审计和追溯操作的重要依据。
- **异常检测与告警**: 监控日志，对异常行为（如短时间内大量失败登录、非常规的数据访问模式）进行告警，及时发现潜在的安全破坏行为。

4.2.2.2 资源保护方式 (Resource Protection)

- **物理安全**: 数据库服务器应放置在安全的机房，限制物理访问。
- **网络隔离**: 数据库服务应部署在受防火墙保护的私有网络中，只允许来自受信任应用服务器的连接。
- **操作系统安全**: 确保数据库服务器的操作系统已打上所有安全补丁，并配置最小权限原则。
- **数据库加固**: 遵循数据库供应商的安全最佳实践，如禁用不必要的服务、修改默认端口、定期进行安全配置审查。
- **数据加密**:
    - **静态数据加密 (Encryption at Rest)**: 数据库文件在存储时进行加密，防止未经授权的物理访问获取数据。
    - **传输中数据加密 (Encryption in Transit)**: 数据库连接应使用SSL/TLS等加密协议，保护数据在客户端和服务器之间传输时的安全。特别是涉及到 password、phone、email 等敏感信息时。

4.2.2.3 存取控制方式 (Access Control Mechanisms)

- **自主存取控制 (Discretionary Access Control, DAC)**:

- **GRANT/REVOKE 语句**：数据库管理员（DBA）使用 GRANT 语句赋予用户或角色对特定数据库对象（表、视图、存储过程等）的特定权限，使用 REVOKE 语句回收权限。这是MySQL等关系型数据库常用的权限管理方式。
- **用户/角色管理**：创建不同的数据库用户或角色，并为它们分配权限，而不是直接对单个用户授权，以简化管理。例如，可以创建“NewsAdmin”角色，授予其对 news 和 carousel_image 表的读写权限。

- **强制存取控制（Mandatory Access Control，MAC）**：
  - 在本项目中，MAC通常不在应用层直接实现，更多是在操作系统或特殊安全数据库中采用。它基于安全级别和分类，不适用于常见的业务数据库设计。
- **基于角色的访问控制（Role-Based Access Control，RBAC）**：
  - 这是本项目最适合采用的存取控制模型。通过 is_admin字段和department字段，可以为用户分配不同的角色。
  - 例如，可以定义以下角色：
    - **系统管理员**：对所有表有完全权限，例如管理 user_info、carousel_image、news的显示状态和排序。
    - **提案管理员**：对 proposal_info、proposal_approval、proposal_file 有管理权限。
    - **会议管理员**：对 meeting_info、meeting_conferee、meeting_file、meeting_summary 有管理权限。
    - **普通用户**：只能发起提案、参与会议、查看新闻和个人信息。
  - proposal_owner 和 meeting_owner 则可以视为特殊的角色权限，允许其对自己创建的记录进行特定操作。

附件 1　数据库逻辑结构字典格式

1.用户服务数据库（user_service）

表名称：user_info

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		id		varchar	20		无	是			是			系统级	用户登录账号，唯一标识一个用户
		password		varchar	30		无				否			系统级	用户登录密码
		name		varchar	20		无				否			用户级	用户真实姓名
		is_admin		tinyint	1		无				否	0，1	0：非管理员，1：管理员	系统级	标识用户是否为管理员
		department	tinyint				无				否	0-9	0：系统管理员，1：校长办公室，2：教务处，3：财政部，4：学工，5：人事部，6：石油学院，	系统级	用户所属部门编码

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
													7：工学院，8：文理学院，9：工商马院		
		phone	varchar	255			无			否				用户级	用户联系电话
		email	varchar	255			无			否				用户级	用户电子邮箱
		Office	varchar	255			无			否				用户级	用户办公地点

2.新闻服务数据库（news-service）

表名称：carousel_image

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		carousel_id		int	-		无	是			是	>0	-	系统级	轮播图的唯一标识符
		Title		varchar	100		无				是	-	-	管理员	轮播图标题
		link_url		varchar	255		无				否	URL格式	-	管理员	点击轮播图后的跳转链接
		sort_order		int	-		无				是	>=0	-	管理员	轮播图的显示排序，数字越小越靠前
		Status		tinyint	-		无				是	0, 1	0: 停用, 1: 启用	管理员	轮播图的启用状态

		created_time		datetime	-		日期时间				是	-	当前时间戳	系统自动	记录创建时间
		updated_time		datetime	-		日期时间				是	-	当前时间戳，更新时刷新	系统自动	最后更新时间
		description		varchar	255		无				是	-	-	管理员	图片描述
		file_path		varchar	255		无				否	-	-	管理员	图片文件路径(用于生成签名URL)

表名：news

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		news_id		int	-		无	是	-	-	是	>0	-	系统级	新闻记录的唯一标识符
		proposal_id		int	-		无	-	proposal_id	proposal_info	是	>0	-	系统级	关联的原提案ID
		proposal_name		varchar	50		无	-	-	-	是	-	-	系统同步	提案标题
		proposal_abstract		text	-		无	-	-	-	否	-	-	系统同步	提案摘要

		proposal_owner		varchar	50		无	-	-	-	是	-	-	系统同步	提案发起人
		proposal_start_time		varchar	255		日期时间字符串	-	-	-	是	YYYY-MM-DD HH	-	系统同步	提交时间
		proposal_type		tinyint	-		无	-	-	-	是	0，1	0：其他，1：重大事项决策	系统同步	提案类型
		proposal_priority		tinyint	-		无	-	-	-	是	2000，1，2	0：紧急，1：较高，2：普通	系统同步	优先级
		display_status		tinyint	1		无	-	-	-	是	0，1	0：停用，1：启用	管理员	是否前端展示
		sort_order		int	-		无		-	-	是	>=0	-	管理员	前端自定义排序
		created_at		datetime	-		日期时间		-	-	是	-	当前时间戳	系统自动	记录创建时间
		updated_at					日期时间		-	-	是	-	当前时间戳，更新时刷新	系统自动	最后更新时间

3.会议服务数据库（meeting_service）

表名: meeting_conferee

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		conferee_id		int			无	是	-	-	是	>0		系统级	参会记录的唯一标识符
		meeting_id		int			无	-	meetint_id	meeting_info	否	>0		系统级	关联会议 ID
		user_id		varchar			无	-	id	user_info	否	-		系统级	参会人员用户 ID

表名: meeting_file

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		file_id		int	-	-	无	是	-	-	是	>0	-	系统级	文件记录的唯一标识符
		file_name		varchar	255	-	无	-	-	-	否	-	-	用户级	文件名称
		meeting_id		int	-	-	无	-	meetint_id	meeting_info	否	>0	-	系统级	关联的会议 ID

		file_type		tinyint	-	-	无	-	-	-	否	0, 1	0: 开会上传材料，1: 会议完成总结材料	用户级	文件类型

表名: meeting_info

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		meetint_id		int	-	-	无	是	-	-	是	>0	-	系统级	会议记录的唯一标识符
		meeting_name		varchar	255	-	无	-	-	-	否	-	-	用户级	会议名称
		meeting_owner		varchar	100	-	无	-	id	user_info	否	-	-	系统级	会议发起人用户 ID
		metting_department		tinyint	-	-	无	-	-	-	否	0-9	同 user_info.department	系统同步	会议发起人所属部门编码
		meeting_content		text	-	-	无	-	-	-	否	-	-	用户级	会议内容详情

		meeting_start_time		varchar	255	-	日期时间字符串	-	-	-	否	-	-	用户级	会议开始时间
		meeting_end_time		varchar	255	-	日期时间字符串	-	-	-	否	-	-	用户级	会议自动结束时间
		proposal_id		int	-	-	无	-	proposal_id	proposal_info	否	>0	-	系统级	关联的提案 ID
		meeting_status		tinyint	-	-	无	-	-	-	否	2000, 1, 2	0: 待开始, 1: 进行中, 2: 完会	系统级	会议状态

		meeting_type		tinyint	-	-	无	-	-	-	否	0-4	0: 其他，1: 重大事项决策，2: 重要干部任免，3: 重大投资项目决策，4: 大额资金使用	用户级	会议类型

表名: meeting_summary

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		summary_id	int	-	-	无	是	-	-	是	>0	-	系统级	会议总结的唯一标识符	int
		meeting_id	int	-	-	无	-	meetint_id	meeting_info	否	>0	-	系统级	关联的会议ID	int
		approval_id	int	-	-	无	-	approval_id	proposal_approval	否	>0	-	系统级	关联的审批ID	int
		summary_content	text	-	-	无	-	-	-	否	-	-	系统生成	AI总结生成内容	text

4. 提案服务数据库（proposal_service）

表名: proposal_approval

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		approval_id		int	-	-	无	是	-	-	是	>0	-	系统级	审批记录的唯一标识符
		proposal_id		int	-	-	无	-	proposal_id	proposal_info	否	>0	-	系统级	关联的提案 ID
		approval_level		tinyint	-	-	级别	-	-	-	否	-	-	系统级	议题审批到第几级了，等同于 proposal_level
		approval_department		tinyint	-	-	无	-	-	-	否	0-9	0: 系统管理员，1: 校长办公室，..., 9: 工商马院	系统级	审批部门编码

		approval_status		tinyint	-	-	无	-	-	-	否	2000, 1, 2	0: 未审批, 1: 同意, 2: 拒绝	系统级	审批状态
		approval_time		varchar	255	-	日期时间字符串	-	-	-	否	yyyy-mm-dd hh	-	系统级	审批时间
		approval_opinion		text	-	-	无	-	-	-	否	-	-	审批人	审批意见
		approval_user		varchar	100	-	无	-	id	user_info	否	-	-	系统级	审批人用户 ID

表名: proposal_file

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		file_id		int	-	-	无	是	-	-	是	>0	-	系统级	文件记录的唯一标识符
		file_name		varchar	255	-	无	-	-	-	否	-	-	用户级	文件名称
		proposal_id		int	-	-	无	-	proposal_id	proposal_info	否	>0	-	系统级	关联的提案 ID

表名: proposal_info

表代码	序号	数据项名称	拼音代码	数据类型	宽度	小数位数	计量单位	主键	外键	参照表	非空值	值约束	规范值	维护级别	数据项描述
		proposal_id		int	-	-	无	是	-	-	是	>0	-	系统级	提案记录的唯一标识符
		proposal_type		tinyint	-	-	无	-	-	-	否	0-4	0: 其他, 1: 重大事项决策, 2: 重要干部任免, 3: 重大投资项目决策, 4: 大额资金使用	用户级	提案类型
		proposal_name		varchar	50	-	无	-	-	-	否	-	-	用户级	提案名称
		proposal_owner		varchar	50	-	无	-	id	user_info	否	-	-	用户级	提案发起人用户 ID
		proposal_abstract		text	-	-	无	-	-	-	否	-	-	用户级	提案摘要

		proposal_start_time		varchar	255	-	日期时间字符串	-	-	-	否	YYYY-MM-DD HH	-	用户级	提案提交时间
		proposal_level		tinyint	-	-	级别	-	-	-	否	-	-	系统级	议题审批到第几级了
		proposal_priority		tinyint	-	-	无	-	-	-	否	2000, 1, 2	0: 紧急, 1: 较高, 2: 普通	用户级	优先级
		proposal_rate		double	-	-	分	-	-	-	否	0.0-5.0 (示例)	-	系统计算	平均评分 (完成了才有)
		proposal_status		tinyint	1	-	无	-	-	-	否	0, 1	0: 审批中, 1: 审批完成	系统级	提案状态
		proposal_end_time		varbinary	255	-	日期时间字符串	-	-	-	否	-	-	系统级	提案结束时间

本数据库设计涵盖了用户、新闻、会议和提案等核心业务，结构清晰、关联明确。用户服务(user_info)表实现了用户身份、权限和部门管理；新闻服务(carousel_image, news)表支持轮播图展示及新闻与提案的关联；会议服务(meeting_info, meeting_conferee, meeting_file, meeting_summary)表实现了会议基本信息、参会人员、会议文件和会议总结的全流程管理，并与用户和提案表建立了外键关联；提案服务(proposal_info, proposal_approval, proposal_file)表实现了提案的基本信息、审批流和相关文件的管理，支持多级审批和状态追踪。整体设计注重数据一致性、可扩展性和业务流程闭环，便于后续系统维护和功能拓展。

本系统的数据库设计符合第三范式（3NF），因为每个表都保证了：每一列都与主键直接相关，没有部分依赖和传递依赖。例如，用户信息、会议、提案、审批、文件等实体分别建表，字段只存储与本表主键直接相关的数据，避免了数据冗余和异常更新；外键用于建立实体间的关联，保证了数据的规范性和一致性。这样设计有助于数据的高效维护和扩展，确保了数据的完整性和逻辑独立性。